Lab 4: Express.js, MongoDB (Mongoose), Validation, Image Upload, and Relationships

Overview

In this lab, you will refactor your Lab 3 REST API. Instead of storing data in JSON files using the File System module, you will use MongoDB with Mongoose. You will also implement validation, image uploads, password hashing, and relationships between collections.

Objectives

- Replace the File System with MongoDB using Mongoose.
- Build CRUD APIs for Users and Posts.
- Create a One-to-Many relationship (One User → Many Posts).
- Validate request data using Joi.
- Validate schemas using Mongoose.
- Upload images to ImageKit.
- Hash user passwords before saving.
- Use environment variables with .env.
- Implement cascading delete for user posts.

Schemas

User:
- name
- email
- password
- age
- image

Post:
- title
- content
- author (ObjectId referencing User)
- image

Required Tasks

1. Create Express project connected to MongoDB using Mongoose.

2. Create User and Post models.

3. Implement CRUD operations for both collections.

4. Define a One-to-Many relationship using ObjectId reference.

5. Before creating a post, verify the referenced user exists.

6. Validate all requests with Joi.

7. Add Mongoose schema validation.

8. Upload user and post images to ImageKit.

9. Store only the ImageKit URL in MongoDB.

10. When deleting a user, automatically delete all posts belonging to that user.

11. Store sensitive configuration in a .env file.

12. Hash user passwords using bcrypt before saving.

13. Handle errors with proper HTTP status codes and JSON responses.

Bonus

- Implement login endpoint using hashed passwords.
- Prevent duplicate emails.
- Add pagination for posts.
- Add search by post title.